

Java 21 Modern Programming & Concurrency — Hands-On Lab Manual

1. Lab Overview

This hands-on laboratory is designed for developers who already understand core Java and want practical experience with modern Java programming techniques available in **Java 21**.

The labs progressively move from functional programming and lambda expressions to advanced concurrency using:

- Functional interfaces
- Lambda expressions
- Method references
- Stream API
- Collectors
- Parallel streams
- Thread and Runnable/Callable
- Thread pools
- Future
- CompletableFuture
- Virtual threads
- ExecutorService
- Structured concurrency
- Concurrency coordination and error handling

Target Audience

- Java developers with basic/intermediate Java knowledge
- Developers moving from traditional Java to modern Java
- Spring Boot and microservices developers
- Developers preparing for Java 21 enterprise development

Prerequisites

Participants should be comfortable with:

- Classes and objects
- Interfaces
- Inheritance
- Exceptions

- Collections
- Generics
- Basic Java I/O
- Basic multithreading concepts

Software Requirements

- JDK 21
- IntelliJ IDEA, Eclipse, or VS Code
- Maven 3.8+
- Git — optional

Verify the installation:

```
java -version
javac -version
mvn -version
```

The Java version should report JDK 21.

2. Lab Scenario

Throughout the exercises, use a common enterprise-style domain.

Create the following classes:

Employee
Department
Project
Customer
Order
Product

The main Employee model can initially contain:

```
public record Employee(  
    int id,  
    String name,  
    String department,  
    double salary,  
    int age,  
    String city  
) {}
```

Create at least 15 employees distributed across departments such as:

IT
HR
Finance
Sales
Operations
Management

Example:

```
List<Employee> employees = List.of(  
    new Employee(101, "Amit", "IT", 85000, 32, "Hyderabad"),  
    new Employee(102, "Priya", "HR", 65000, 29, "Bangalore"),  
    new Employee(103, "Rahul", "IT", 95000, 35, "Pune"),  
    new Employee(104, "Sneha", "Finance", 78000, 31, "Mumbai"),  
    new Employee(105, "Arjun", "Sales", 72000, 28, "Delhi"),  
    new Employee(106, "Neha", "IT", 110000, 40, "Hyderabad"),  
    new Employee(107, "Vikram", "Operations", 68000, 36, "Chennai"),  
    new Employee(108, "Kavita", "HR", 71000, 34, "Pune"),  
    new Employee(109, "Ravi", "Finance", 88000, 39, "Hyderabad"),  
    new Employee(110, "Anita", "Sales", 92000, 37, "Bangalore"),  
    new Employee(111, "Suresh", "IT", 76000, 27, "Chennai"),  
    new Employee(112, "Meena", "Operations", 82000, 33, "Mumbai"),  
    new Employee(113, "Kiran", "Finance", 99000, 42, "Delhi"),  
    new Employee(114, "Pooja", "Sales", 69000, 26, "Pune"),  
    new Employee(115, "Manoj", "Management", 135000, 48, "Hyderabad")  
);
```

LAB 1 — Functional Programming with Java

Objective

Understand how Java supports functional programming while remaining an object-oriented language.

Concepts

Practice:

- Pure functions
- Functions as values
- Immutability
- Higher-order functions
- Functional interfaces
- Lambda expressions
- Method references

Exercise 1.1 — Traditional vs Functional Approach

Create a method that calculates the square of a number.

Traditional:

```
static int square(int n) {  
    return n * n;  
}
```

Create a functional version using:

```
Function<Integer, Integer>
```

Expected usage:

```
Function<Integer, Integer> square = ...;
```

```
System.out.println(square.apply(10));
```

Expected output:

```
100
```

Exercise 1.2 — Function Composition

Create:

```
Function<Integer, Integer> multiplyByTwo;  
Function<Integer, Integer> addTen;
```

Compose them using:

```
andThen()  
compose()
```

Experiment with:

```
multiplyByTwo.andThen(addTen)
```

and:

```
multiplyByTwo.compose(addTen)
```

Observe the difference.

Exercise 1.3 — Predicate

Create a predicate that determines whether an employee earns more than ₹80,000.

```
Predicate<Employee> highSalary = ...;
```

Use:

```
highSalary.test(employee)
```

Extend the exercise with:

```
Predicate<Employee> experienced;  
Predicate<Employee> fromHyderabad;
```

Combine predicates using:

```
and()  
or()  
negate()
```

Exercise 1.4 — Consumer

Create:

```
Consumer<Employee>
```

that prints:

```
Employee: Amit, Salary: 85000
```

Apply it to every employee.

Exercise 1.5 — Supplier

Create a:

```
Supplier<Employee>
```

that generates a default employee.

Challenge

Create a method:

```
static void processEmployees(  
    List<Employee> employees,  
    Predicate<Employee> condition,  
    Consumer<Employee> action)
```

The method should process only employees satisfying the supplied condition.

Example:

```
processEmployees(  
    employees,  
    e -> e.salary() > 80000,  
    e -> System.out.println(e.name())  
);
```

LAB 2 — Lambda Expressions

Objective

Master lambda syntax and understand where lambda expressions can be used.

Exercise 2.1 — Lambda Syntax

Create lambdas for:

```
Runnable  
Comparator<Employee>  
Predicate<Employee>  
Function<Employee, String>  
Consumer<Employee>  
Supplier<Employee>
```

Exercise 2.2 — Sorting

Sort employees by salary.

First use an anonymous class.

Then replace it with a lambda.

Finally use:

```
Comparator.comparing(...)
```

Exercise 2.3 — Multiple Conditions

Sort employees by:

1. Department
2. Salary descending
3. Name ascending

Use:

```
Comparator.comparing(...)  
            .thenComparing(...)  
            .thenComparing(...)
```

Exercise 2.4 — Method References

Replace:

```
e -> e.name()
```

with a method reference where possible.

Experiment with:

```
Employee::name  
System.out::println  
String::toUpperCase
```

LAB 3 — Stream Processing

Objective

Use the Java Stream API to perform enterprise-style collection processing.

This lab should be treated as the **major Stream API laboratory**.

Part A — Creating Streams

Practice creating streams using:

```
employees.stream()
```

`Stream.of(...)`

`Arrays.stream(...)`

`Stream.generate(...)`

`Stream.iterate(...)`

Part B — Filtering

Find:

Exercise 3.1

Employees earning more than ₹80,000.

```
employees.stream()  
    .filter(...)
```

Exercise 3.2

Employees from IT.

Exercise 3.3

Employees from Hyderabad earning more than ₹80,000.

Exercise 3.4

Employees who are not from HR.

Use:

```
filter()
```

and:

```
Predicate.negate()
```

Part C — Mapping

Exercise 3.5

Print employee names only.

```
map(Employee::name)
```


Exercise 3.6

Create a stream containing employee salaries.

Exercise 3.7

Convert names to uppercase.

Exercise 3.8

Convert employees into:

Employee Name -> Department

Part D — flatMap

Create:

```
List<List<String>> skills
```

Example:

```
List<List<String>> employeeSkills = List.of(  
    List.of("Java", "Spring"),  
    List.of("Kafka", "Docker"),  
    List.of("Java", "AWS")  
);
```

Flatten it using:

```
flatMap()
```

Expected result:

```
Java  
Spring  
Kafka  
Docker  
Java  
AWS
```

Part E — Sorting

Implement:

Exercise 3.9

Sort employees by salary ascending.

Exercise 3.10

Sort employees by salary descending.

Exercise 3.11

Sort by name.

Exercise 3.12

Sort by department and then salary.

Exercise 3.13

Sort by:

Department ascending

Salary descending

Name ascending

Use:

```
Comparator.comparing()  
Comparator.comparingDouble()  
thenComparing()  
reversed()
```

Part F — Distinct

Find unique departments.

```
employees.stream()  
    .map(Employee::department)  
    .distinct()
```

Part G — Limit and Skip

Exercise 3.14

Find the top five highest-paid employees.

Use:

```
sorted()  
limit()
```

Exercise 3.15

Find employees ranked 6–10 by salary.

Use:

```
skip()  
limit()
```

Part H — takeWhile and dropWhile

Sort employees by salary and experiment with:

```
takeWhile()
```

and:

```
dropWhile()
```

Understand why these operations are especially meaningful when the stream has an appropriate encounter order.

Part I — Matching Operations

Implement:

```
anyMatch()  
allMatch()  
noneMatch()
```

Answer:

1. Is anyone earning more than ₹150,000?
 2. Does everyone earn more than ₹50,000?
 3. Is nobody from the Legal department?
 4. Does anyone work in Hyderabad?
-

Part J — Finding Elements

Practice:

`findFirst()`
`findAny()`

Find:

- First IT employee
 - Any employee earning over ₹100,000
 - First employee after sorting by salary
-

Part K — Counting

Use:

`count()`

Find:

- Number of employees
 - Number of IT employees
 - Number of employees earning more than ₹80,000
-

Part L — Numeric Streams

Convert salaries into:

`IntStream`
`LongStream`
`DoubleStream`

Practice:

`mapToInt()`
`mapToLong()`
`mapToDouble()`

Calculate:

`Sum`
`Average`
`Minimum`

Maximum
Count

Part M — Reduction

Use:

`reduce()`

Calculate:

Total salary
Maximum salary
Minimum salary
Product of numbers

Example concept:

```
numbers.stream()  
    .reduce(...)
```

Experiment with both:

`reduce(identity, accumulator)`

and:

`reduce(identity, accumulator, combiner)`

Discuss why the third form matters for parallel streams.

Part N — Collectors

Practice:

`collect()`

with:

```
Collectors.toList()  
Collectors.toSet()  
Collectors.toMap()  
Collectors.joining()  
Collectors.counting()  
Collectors.summingInt()  
Collectors.summingDouble()
```

```
Collectors.averagingDouble()  
Collectors.minBy()  
Collectors.maxBy()
```

Part O — Grouping

Exercise 3.16 — Group by Department

```
Collectors.groupingBy(Employee::department)
```

Expected conceptual structure:

```
IT          -> [...]  
HR          -> [...]  
Finance     -> [...]  
Sales       -> [...]
```

Exercise 3.17 — Count Employees by Department

Use:

```
groupingBy(  
    Employee::department,  
    counting()  
)
```

Exercise 3.18 — Average Salary by Department

Use:

```
averagingDouble()
```

Exercise 3.19 — Maximum Salary by Department

Use:

```
maxBy()
```

Exercise 3.20 — Minimum Salary by Department

Use:

`minBy()`

Part P — groupingBy with Mapping

Create:

Department -> List of Employee Names

Use:

`groupingBy()`
`mapping()`

Part Q — Partitioning

Partition employees into:

`salary > 80000`
`salary <= 80000`

Use:

`partitioningBy()`

Then create:

Department -> High Salary Count

Part R — Joining

Produce:

Amit, Priya, Rahul, Sneha, ...

Use:

`Collectors.joining(", ")`

Part S — toMap

Create:

Employee ID -> Employee

using:

`Collectors.toMap()`

Intentionally create duplicate keys and observe the exception.

Then solve it using a merge function.

Part T — Advanced Collector Exercise

Produce:

Department

- |
- +-- Employee count
- +-- Average salary
- +-- Maximum salary
- +-- Employee names

Use nested collectors such as:

```
groupingBy()
collectingAndThen()
mapping()
counting()
averagingDouble()
maxBy()
```

LAB 4 — Parallel Stream Processing

Objective

Understand parallel stream execution and its appropriate use cases.

Exercise 4.1

Create a large list:


```
List<Integer> numbers
```

containing at least 10 million integers.

Compare:

```
numbers.stream()
```

with:

```
numbers.parallelStream()
```

Measure execution time.

Exercise 4.2

Print the executing thread:

```
System.out.println(  
    Thread.currentThread().getName()  
);
```

Observe the difference.

Exercise 4.3

Compare:

```
for loop  
stream  
parallelStream
```

for a CPU-intensive operation.

Exercise 4.4 — Parallel Stream Trap

Create a mutable:

```
ArrayList<Integer>
```

and modify it from a parallel stream.

Observe the result.

Explain why the following is unsafe:

```
List<Integer> result = new ArrayList<>();
```

```
numbers.parallelStream()  
    .forEach(result::add);
```

Replace it with an appropriate collector.

LAB 5 — Basic Java Concurrency

Objective

Understand the Java concurrency programming model before moving to executors and virtual threads.

Exercise 5.1 — Creating Threads

Create a thread using:

`Thread`

and:

`Runnable`

Run multiple tasks concurrently.

Exercise 5.2 — Lambda Runnable

Replace:

```
new Runnable() {  
    @Override  
    public void run() {  
        ...  
    }  
}
```

with:

```
Runnable task = () -> {  
    ...  
};
```

Exercise 5.3 — Thread Lifecycle

Experiment with:

```
start()
join()
sleep()
interrupt()
isAlive()
```

Observe thread state transitions.

LAB 6 — Race Conditions and Synchronization

Objective

Understand shared mutable state and race conditions.

Exercise 6.1

Create:

```
class Counter {
    private int count;

    public void increment() {
        count++;
    }
}
```

Run 100 threads, each incrementing the counter 1,000 times.

Expected:

100000

Observe the actual result.

Exercise 6.2

Fix the problem using:

synchronized

Exercise 6.3

Fix it using:

`AtomicInteger`

Compare the approaches.

Exercise 6.4

Experiment with:

`ReentrantLock`

Discuss:

- Locking
 - Unlocking
 - `tryLock()`
 - Fairness
 - Exception safety
-

LAB 7 — Thread Pools and ExecutorService

Objective

Understand why enterprise applications generally use executors rather than manually creating threads.

Exercise 7.1 — Fixed Thread Pool

Create:

```
ExecutorService executor =  
    Executors.newFixedThreadPool(5);
```

Submit 20 tasks.

Print:

Task ID
Thread name
Start time
End time

Observe thread reuse.

Exercise 7.2 — Cached Thread Pool

Replace the executor with:

```
Executors.newCachedThreadPool()
```

Compare behavior.

Exercise 7.3 — Scheduled Executor

Use:

```
ScheduledExecutorService
```

to:

1. Execute a task after 5 seconds.
 2. Execute a task periodically.
 3. Cancel the task.
-

Exercise 7.4 — Graceful Shutdown

Implement:

```
shutdown()  
awaitTermination()  
shutdownNow()
```

Observe the differences.

LAB 8 — Callable and Future

Objective

Learn how to submit tasks that return results.

Exercise 8.1

Create:

`Callable<Integer>`

that calculates the sum of 1 to 1,000,000.

Submit it using:

`executor.submit()`

Retrieve the result using:

`future.get()`

Exercise 8.2 — Multiple Futures

Submit five employee-processing tasks.

Each task should return:

Department -> Employee Count

Collect the results.

Exercise 8.3 — Timeout

Use:

`future.get(2, TimeUnit.SECONDS)`

Create a task that sleeps for five seconds.

Observe:

`TimeoutException`

Exercise 8.4 — Cancellation

Start a long-running task.

Cancel it using:

`future.cancel(true)`

Check:

```
future.isCancelled()  
future.isDone()
```

LAB 9 — CompletableFuture

Objective

Build asynchronous pipelines using CompletableFuture.

Exercise 9.1 — Supply Async

Create:

```
CompletableFuture.supplyAsync(...)
```

Return an employee.

Retrieve the result using:

```
join()
```

Exercise 9.2 — Transformation

Build:

```
Employee  
  ↓  
Employee Name  
  ↓  
Uppercase Name
```

Use:

```
thenApply()
```

Exercise 9.3 — Consumer

Use:

`thenAccept()`

to print the result.

Exercise 9.4 — Runnable Completion

Use:

`thenRun()`

to execute a final action.

Exercise 9.5 — Chaining

Create:

```
getEmployee()  
  ↓  
getDepartment()  
  ↓  
getManager()  
  ↓  
generateReport()
```

Implement the pipeline using:

`thenCompose()`

Exercise 9.6 — thenApply vs thenCompose

Create a method:

`CompletableFuture<Employee> findEmployee(int id)`

and:

`CompletableFuture<Department> findDepartment(Employee employee)`

Demonstrate why:

`thenApply()`

and:

`thenCompose()`

produce different types.

Exercise 9.7 — Combining Independent Operations

Simulate three remote services:

Customer Service

Order Service

Payment Service

Each service should sleep for a different amount of time.

Execute them concurrently.

Use:

`CompletableFuture.allOf()`

Exercise 9.8 — first Completed

Simulate three services:

Service A -> 3 seconds

Service B -> 1 second

Service C -> 2 seconds

Use:

`CompletableFuture.anyOf()`

Return the first response.

Exercise 9.9 — Error Handling

Create a pipeline that deliberately throws an exception.

Experiment with:

```
exceptionally()  
handle()  
whenComplete()
```

Compare their behavior.

Exercise 9.10 — Custom Executor

Do not use the common ForkJoinPool.

Create:

```
ExecutorService executor =  
    Executors.newFixedThreadPool(10);
```

Pass it to:

```
CompletableFuture.supplyAsync(  
    task,  
    executor  
);
```

Discuss why explicitly controlling the executor can be important in enterprise applications.

LAB 10 — Virtual Threads

Objective

Understand Project Loom's virtual threads and how they differ from traditional platform threads.

Virtual threads became a permanent Java feature in Java 21.

Exercise 10.1 — Create a Virtual Thread

Create:

```
Thread.startVirtualThread(() -> {  
    System.out.println(  
        Thread.currentThread()  
    );  
})
```

```
});
```

Observe the thread information.

Exercise 10.2 — Virtual Thread Builder

Use:

```
Thread.ofVirtual()  
    .name("worker")  
    .start(task);
```

Exercise 10.3 — Thousands of Virtual Threads

Create 10,000 tasks.

Each task should:

```
Thread.sleep(1000);
```

Execute them using virtual threads.

Observe memory usage and execution behavior.

Exercise 10.4 — Platform vs Virtual Threads

Compare:

```
Thread.ofPlatform()
```

with:

```
Thread.ofVirtual()
```

Create a large number of each where practical and compare:

- Creation cost
- Memory consumption
- Scheduling
- Execution

- Blocking behavior
-

Exercise 10.5 — Virtual Thread Executor

Use:

`Executors.newVirtualThreadPerTaskExecutor()`

Submit 1,000 tasks.

Example:

```
try (var executor =
    Executors.newVirtualThreadPerTaskExecutor()) {

    for (int i = 0; i < 1000; i++) {
        int taskId = i;

        executor.submit(() -> {
            System.out.println(
                "Task " + taskId +
                " : " +
                Thread.currentThread()
            );

            Thread.sleep(1000);
            return taskId;
        });
    }
}
```

Exercise 10.6 — Simulated Microservice Calls

Create:

Customer Service
Inventory Service
Payment Service
Shipping Service

Each call should simulate network latency using:

`Thread.sleep()`

Execute thousands of independent requests using virtual threads.

Discuss why virtual threads are particularly useful for workloads involving blocking I/O.

Exercise 10.7 — Virtual Threads and Thread Pools

Compare these approaches:

```
Executors.newFixedThreadPool(20)
```

and:

```
Executors.newVirtualThreadPerTaskExecutor()
```

for an I/O-bound workload.

Important discussion point:

Virtual threads are not a replacement for every use of a bounded executor. CPU-intensive work still needs appropriate concurrency limits.

LAB 11 — Virtual Thread Server Simulation

Scenario

Simulate an HTTP server receiving 10,000 requests.

Each request performs:

```
Request
  ↓
Authenticate
  ↓
Database call
  ↓
REST service call
  ↓
Response
```

Each external operation should be simulated with:

```
Thread.sleep()
```

Implement the server using:

Version A

Fixed thread pool.

Version B

Virtual threads.

Measure:

Total requests

Successful requests

Failed requests

Total execution time

Average execution time

LAB 12 — Structured Concurrency

Objective

Understand structured concurrency in Java 21.

Important Java 21 Note

Structured concurrency was introduced as a **preview feature** in Java 21.

Therefore, Java 21 programs using the Java 21 structured concurrency API require preview support.

Compile with:

```
javac --enable-preview --release 21 YourClass.java
```

Run with:

```
java --enable-preview YourClass
```

For Maven, configure the compiler and JVM execution appropriately with preview enabled.

Exercise 12.1 — First Structured Task

Explore:

StructuredTaskScope

Create two independent tasks:

Get Customer

Get Orders

Execute them concurrently.

Exercise 12.2 — ShutdownOnFailure

Use:

`StructuredTaskScope.ShutdownOnFailure`

Create:

Customer lookup

Order lookup

Payment lookup

If one operation fails, cancel the remaining operations.

Observe the behavior.

Exercise 12.3 — ShutdownOnSuccess

Create three simulated services:

Service A

Service B

Service C

Each returns the same type of information but takes a different amount of time.

Use:

`StructuredTaskScope.ShutdownOnSuccess`

Return the first successful result.

Exercise 12.4 — Parent/Child Task Relationship

Create a parent operation:

Generate Customer Dashboard

It launches:

Customer Details
Orders
Payments
Recommendations

Each operation should be a child task.

Draw the execution structure:

```
Customer Dashboard
  |
  +---- Customer Details
  |
  +---- Orders
  |
  +---- Payments
  |
  +---- Recommendations
```

Discuss how structured concurrency makes task lifetime and ownership explicit.

LAB 13 — CompletableFuture vs Structured Concurrency

Objective

Implement the same business requirement using both APIs.

Scenario

Create a customer dashboard requiring:

Customer
Orders
Payments
Recommendations

Version A — CompletableFuture

Implement using:

CompletableFuture

Use:

`allOf()`

where appropriate.

Version B — Structured Concurrency

Implement the same functionality using:

`StructuredTaskScope`

Compare

Document the differences in:

Area	CompletableFuture	Structured Concurrency
Task composition		
Cancellation		
Error propagation		
Task ownership		
Readability		
Lifecycle management		
Result collection		
Debugging		

LAB 14 — Enterprise Concurrent Application

Final Capstone

Build a small **Order Processing Engine**.

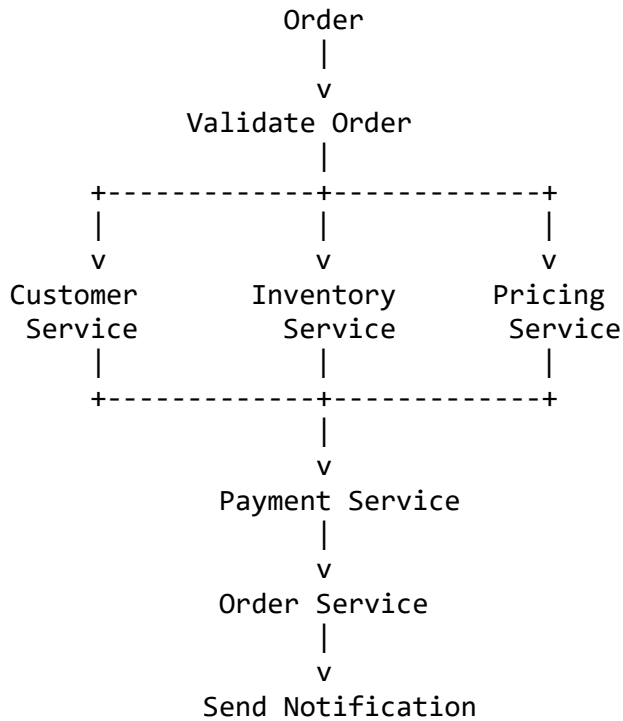
The application receives:

Order

containing:

orderId
customerId
productIds
quantity

The processing pipeline is:



Part A — Functional Programming

Use:

Predicate

Function

Consumer

Supplier

for validation and processing.

Part B — Stream Processing

Use streams to:

- Validate order items
- Calculate total quantity
- Calculate total price
- Group items by product category
- Find the most expensive product
- Sort products by price

- Generate a product summary
-

Part C — Traditional Thread Pool

Implement service calls using:

`ExecutorService`

with a fixed thread pool.

Part D — Future

Return service results using:

`Future`

Part E — CompletableFuture

Replace the previous implementation with:

`CompletableFuture`

Execute independent calls concurrently.

Part F — Virtual Threads

Replace the executor with:

`Executors.newVirtualThreadPerTaskExecutor()`

Compare the implementation and performance.

Part G — Structured Concurrency

Implement the service aggregation using:

`StructuredTaskScope`

Implement failure handling so that if a critical service fails, related tasks are cancelled.

LAB 15 — Performance Comparison

Create a final benchmark-style experiment.

Process 10,000 simulated customer requests.

Each request performs multiple blocking operations.

Compare:

1. Sequential
2. Platform threads
3. Fixed thread pool
4. `CompletableFuture`
5. Virtual threads
6. Structured concurrency

Record:

Total execution time
Average execution time
Maximum execution time
Successful requests
Failed requests
Number of threads

Create a final report explaining the observations.

3. Stream API Operator Checklist

The following operations should be covered during the Stream laboratory.

Intermediate Operations

`filter()`
`map()`
`mapToInt()`
`mapToLong()`
`mapToDouble()`
`flatMap()`
`flatMapToInt()`
`flatMapToLong()`
`flatMapToDouble()`

distinct()
sorted()
peek()
limit()
skip()
takeWhile()
dropWhile()

Terminal Operations

forEach()
forEachOrdered()
toArray()
reduce()
collect()
min()
max()
count()
anyMatch()
allMatch()
noneMatch()
findFirst()
findAny()

Important Collectors

toList()
toSet()
toMap()
joining()
counting()
summarizingInt()
summarizingLong()
summarizingDouble()
summingInt()
summingLong()
summingDouble()
averagingInt()
averagingLong()
averagingDouble()
minBy()
maxBy()
groupingBy()
groupingByConcurrent()
partitioningBy()
mapping()
flatMap()
filtering()
collectingAndThen()
teeing()

Participants should implement examples using each of these where applicable.

4. Concurrency Concepts Checklist

By the end of the labs, participants should be able to explain and demonstrate:

- Thread creation
- Runnable
- Callable
- Thread lifecycle
- Thread interruption
- `join()`
- Race conditions
- Synchronization
- Locks
- Atomic variables
- `ExecutorService`
- Fixed thread pools
- Cached thread pools
- Scheduled executors
- Future
- Future cancellation
- Future timeout
- `CompletableFuture`
- `thenApply()`
- `thenCompose()`
- `thenAccept()`
- `thenRun()`
- `allOf()`
- `anyOf()`
- Exception handling
- Custom executors
- Platform threads
- Virtual threads
- Virtual thread executors
- Blocking I/O with virtual threads
- Structured concurrency

- Task ownership
 - Task cancellation
 - Failure propagation
 - Shutdown policies
-

5. Final Challenge Questions

After completing all labs, answer the following.

Question 1

When should you prefer:

`stream()`

over:

`parallelStream()`

?

Question 2

Why is this potentially unsafe?

```
parallelStream()  
    .forEach(list::add);
```

Question 3

What is the difference between:

`map()`

and:

`flatMap()`

?

Question 4

What is the difference between:

`thenApply()`

and:

`thenCompose()`

?

Question 5

What problem does `CompletableFuture` solve compared with `Future`?

Question 6

Why are virtual threads particularly useful for I/O-bound workloads?

Question 7

Are virtual threads faster than platform threads for CPU-intensive calculations?

Explain.

Question 8

Why should an application avoid blindly creating a huge number of platform threads?

Question 9

What problem does structured concurrency attempt to solve?

Question 10

Compare:

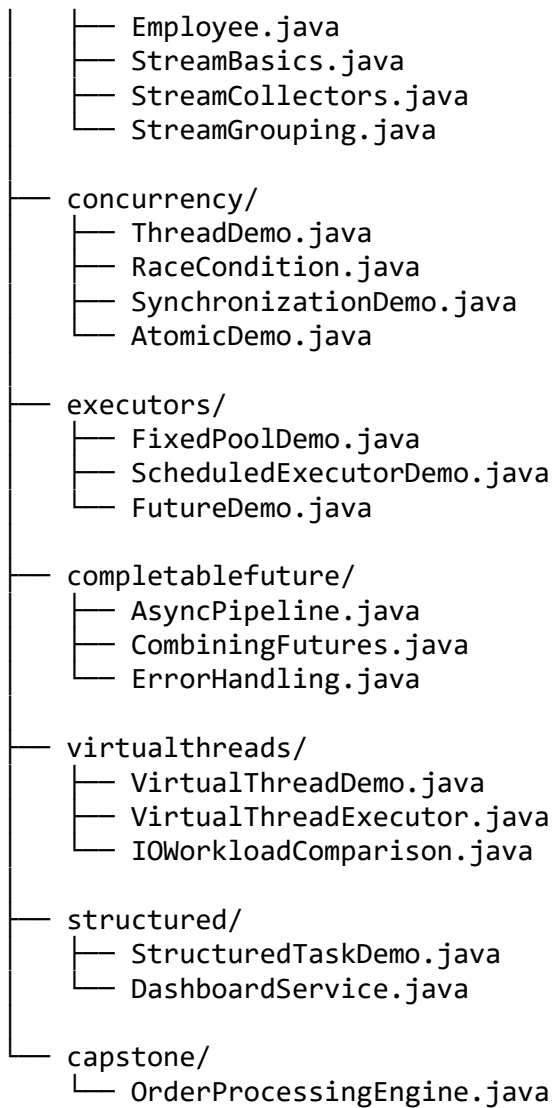
`Thread`
`ExecutorService`
`Future`
`CompletableFuture`
`Virtual Thread`
`StructuredTaskScope`

in terms of abstraction and use case.

6. Suggested Final Project Deliverables

Each participant should submit:

```
java21-modern-labs/  
├── functional/  
│   ├── FunctionalInterfaces.java  
│   └── LambdaExamples.java  
└── streams/
```

7. Expected Learning Outcome

After completing these laboratories, participants should be able to build modern Java 21 applications using functional programming and concurrent programming techniques.

In particular, participants should be able to make informed choices between:

Imperative programming
↓
Functional programming
↓
Stream processing
↓
Executor-based concurrency

